

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №2
по дисциплине «Вычислительная математика»

Вариант: 20

Преподаватель:
Бострикова Дарья Константиновна

Выполнил: Ясаков Артем
Группа: P3213

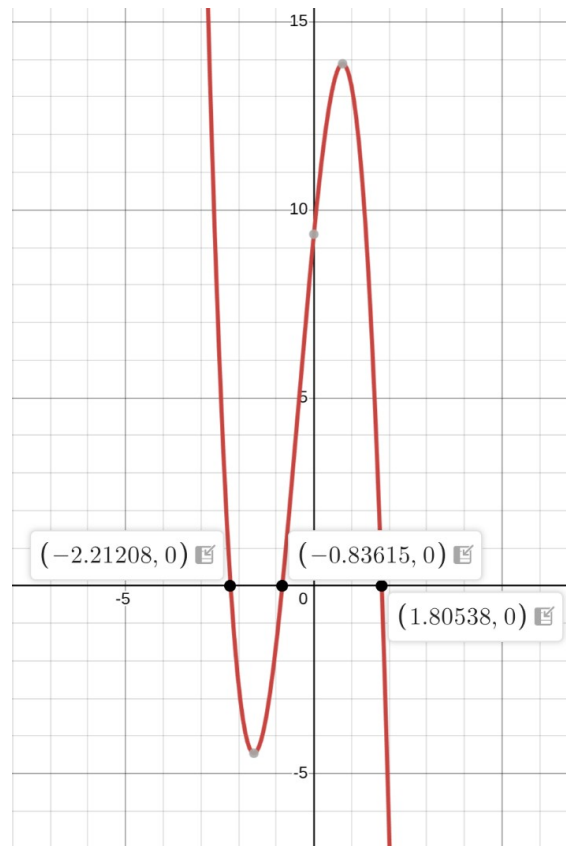
Санкт-Петербург, 2026 г

Цель работы: изучить численные методы решения нелинейных уравнений и их систем, найти корни заданного нелинейного уравнения/системы нелинейных уравнений, выполнить программную реализацию методов.

1. Вычислительная реализация задачи

1. Решение нелинейного уравнения

1. $-2,8x^3 - 3,48x^2 + 10,23x + 9,35$



2.

Для определения интервалов изоляции корней данного уравнения, можно воспользоваться методом интервалов знакопеременности. Для этого нужно найти значения функции на различных интервалах и определить знак функции на каждом из них.

Получим приближенные значения корней:

$$x \approx -2.2, x \approx -0.8, x \approx 1.8$$

Теперь нужно разбить ось x на 4 интервала: $(-\infty, -2.2)$, $(-2.2, -0.8)$, $(-0.8, 1.8)$ и $(1.8, +\infty)$. На каждом из этих интервалов нужно определить знак функции.

Для этого можем вычислить значения функции в произвольной точке каждого интервала.

Например, для интервала $(-\infty, -2.2)$ можно выбрать $x = -3$, для интервала $(-2.2, -0.8)$ $x = -1$, для интервала $(-0.8, 1.8)$ $x = 0$, и для интервала $(1.8, +\infty)$ $x = 2$.

Таким образом, получим следующие значения функции:

для $x = -3$: $f(-4) = 22.94$

для $x = -1$: $f(-2) = -1.56$

для $x = 0$: $f(0) = 9.35$

для $x = 2$: $f(2) = -6,51$

Знаки функции на каждом интервале будут соответственно:

$(-\infty, -2.2)$	$(-2.2, -0.8)$	$(-0.8, 1.8)$	$(1.8, +\infty)$
+	-	+	-

Таким образом, мы получаем следующие интервалы изоляции корней уравнения:

$(-3, -1)$, $(-1, 0)$ и $(0, 2)$.

3.

$x_1 \approx -2.21$

$x_2 \approx -0.84$

$x_3 \approx 1.81$

4.

Крайний правый корень – **Метод половинного деления**

№	a	b	x	f(a)	f(b)	f(x)	a – b
1	0.000	2.000	1.000	9.350	-6.510	13.300	2.000
2	1.000	2.000	1.500	13.300	-6.510	7.415	1.000
3	1.500	2.000	1.750	7.415	-6.510	1.589	0.500
4	1.750	2.000	1.875	1.589	-6.510	-2.160	0.250
5	-1.500	1.875	1.812	1.589	-2.160	-0.213	0.125
6	1.750	1.812	1.781	1.589	-0.213	0.706	0.062
7	1.781	1.812	-1.797	0.706	-0.213	0.251	0.031
8	1.797	1.812	-1.805	0.251	-0.213	0.020	0.016
9	1.805	1.812	1.809	0.020	-0.213	-0.096	0.008

Крайний левый корень – **Метод Ньютона**

№	x	f(x _k)	f'(x _k)	x _{k+1}	x _{k+1} - x _k
1	-3.000	22.940	-44.490	-2.484	0.516
2	-2.484	5.391	-24.325	-2.263	0.222
3	-2.263	0.824	-17.030	-2.214	0.048
4	-2.214	0.036	-15.548	-2.212	0.002

Центральный корень – Метод простой итерации

Проверка условия сходимости метода на выбранном интервале $(-1, 0)$:

$$f(x) = -2,8x^3 - 3,48x^2 + 10,23x + 9,35 = 0$$

$$f'(x) = -8,4x^2 - 6,96x + 10,23$$

$$f'(a) = 8,79 > 0, f'(b) = 10,23 > 0$$

$$\max(f'(a), f'(b)) = 10,23 \rightarrow \lambda = \frac{-1}{\max(f'(x))} = \frac{-1}{10,23}$$

$$\varphi(x) = x + \lambda f(x) = x - \frac{-2,8x^3 - 3,48x^2 + 10,23x + 9,35}{10,23}$$

$$\varphi'(x) = 1 + \lambda f'(x) = 1 - \frac{-8,4x^2 - 6,96x + 10,23}{10,23}$$

На отрезке начального приближения $[-1, 0]$ функция $\varphi(x)$ определена, непрерывна и дифференцируема.

$$|\varphi'(a)| = 0,141$$

$$|\varphi'(b)| = 0$$

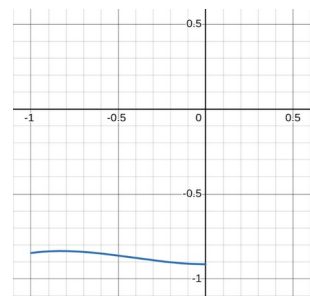
$$|\varphi'(x)| \leq q, \text{ где } q = 0,141$$

$0 \leq q < 1 \rightarrow$ итерационная последовательность сходится, скорость сходимости высокая,

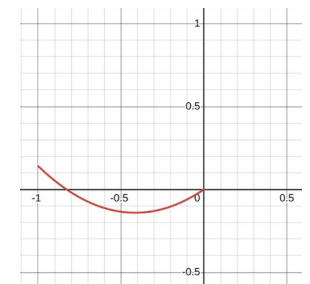
$0 \leq q < 0,5 \rightarrow$ критерий окончания итерационного процесса $|x_{k+1} - x_k| \leq \varepsilon, x_0 = -1.000$

№	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	-1.0000	-0.8475	-0.115035	0.1525
2	-0.8475	-0.8363	-0.00151535	0.0112
3	-0.8363	-0.8362	-0.000497748	0.0001

$\varphi(x)$



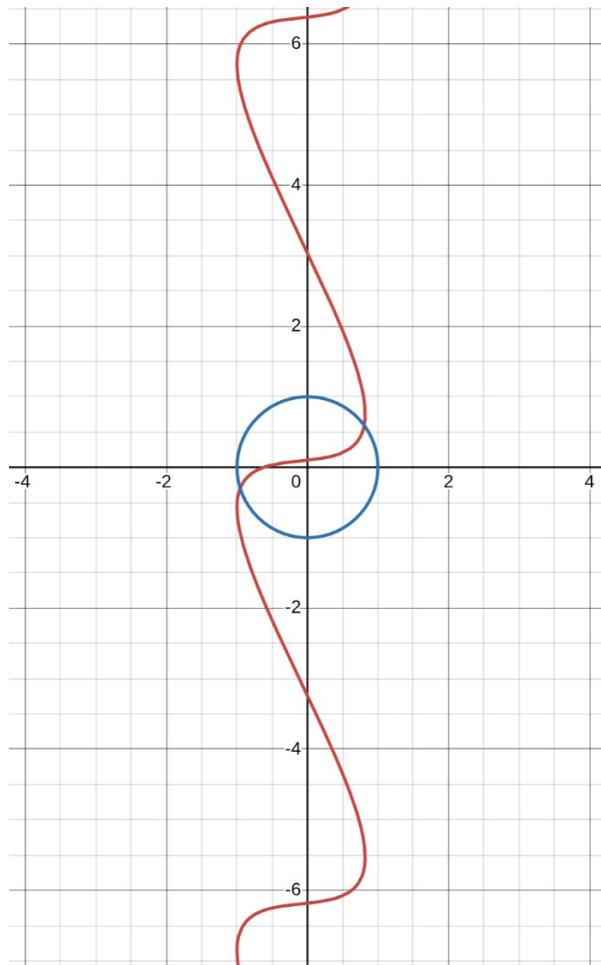
$\varphi'(x)$



2. Решение системы нелинейных уравнений

1. $\begin{cases} \sin(x+y) - 1.1x = 0.1 \\ x^2 + y^2 = 1 \end{cases}$, Метод Ньютона

2.



$$\begin{cases} \sin(x+y) - 1.1x = 0.1 \\ x^2 + y^2 = 1 \end{cases} \rightarrow \begin{cases} f(x,y) = 0 \\ g(x,y) = 0 \end{cases} \rightarrow \begin{cases} \sin(x+y) - 1.1x - 0.1 = 0 \\ x^2 + y^2 - 1 = 0 \end{cases}$$

Построим матрицу Якоби:

$$\frac{\partial f}{\partial x} = \cos(x+y) - 1.1, \quad \frac{\partial f}{\partial y} = \cos(x+y), \quad \frac{\partial g}{\partial x} = 2x, \quad \frac{\partial g}{\partial y} = 2y$$

$$\begin{vmatrix} \frac{\partial f(x,y)}{\partial x} & \frac{\partial f(x,y)}{\partial y} \\ \frac{\partial g(x,y)}{\partial x} & \frac{\partial g(x,y)}{\partial y} \end{vmatrix} \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = - \begin{pmatrix} f(x,y) \\ g(x,y) \end{pmatrix}$$

$$\begin{vmatrix} \cos(x+y) - 1.1 & \cos(x+y) \\ 2x & 2y \end{vmatrix} \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = \begin{pmatrix} 1.1x - \sin(x+y) + 0.1 \\ 1 - x^2 - y^2 \end{pmatrix}$$

$$\begin{cases} (\cos(x+y) - 1.1)\Delta x + \cos(x+y)\Delta y = 1.1x - \sin(x+y) + 0.1 \\ 2x\Delta x + 2y\Delta y = 1 - x^2 - y^2 \end{cases}$$

Корень 1: Шаг 1: Выбираем $x_0=0.8$; $y_0=0.6$

$$\begin{cases} (\cos(x+y) - 1.1)\Delta x + \cos(x+y)\Delta y = 1.1x - \sin(x+y) + 0.1 \\ 2x\Delta x + 2y\Delta y = 1 - x^2 - y^2 \end{cases}$$

Шаг 2. Решаем полученную систему.

$$\begin{cases} -0.93003\Delta x + 0.16997\Delta y = -0.00545 \\ 1.6\Delta x + 1.2\Delta y = 0 \end{cases} \rightarrow \Delta x = 0.00515; \Delta y = -0.00387$$

Шаг 3. Вычисляем очередные приближения:

$$x_1 = x_0 + \Delta x = 0.8 + 0.00515 = 0.80515$$

$$y_1 = y_0 + \Delta y = 0.6 - 0.00387 = 0.59613$$

$$|x_1 - x_0| \leq \varepsilon, |y_1 - y_0| \leq \varepsilon$$

$$|0.80515 - 0.8| \leq \varepsilon, |0.59613 - 0.6| \leq \varepsilon \rightarrow \text{ответ найден, корень 1: } (0.80515, 0.59613)$$

Корень 2: Шаг 1: Выбираем $x_0=-0.96$; $y_0=-0.28$

$$\begin{cases} (\cos(x+y) - 1.1)\Delta x + \cos(x+y)\Delta y = 1.1x - \sin(x+y) + 0.1 \\ 2x\Delta x + 2y\Delta y = 1 - x^2 - y^2 \end{cases}$$

Шаг 2. Решаем полученную систему.

$$\begin{cases} -0.7752\Delta x + 0.3245\Delta y = -0.0102 \\ -1.92\Delta x - 0.56\Delta y = 0 \end{cases} \rightarrow \Delta x = 0.0054; \Delta y = -0.0185$$

Шаг 3. Вычисляем очередные приближения:

$$x_1 = x_0 + \Delta x = -0.96 + 0.0054 = -0.9546$$

$$y_1 = y_0 + \Delta y = -0.28 - 0.0185 = -0.2985$$

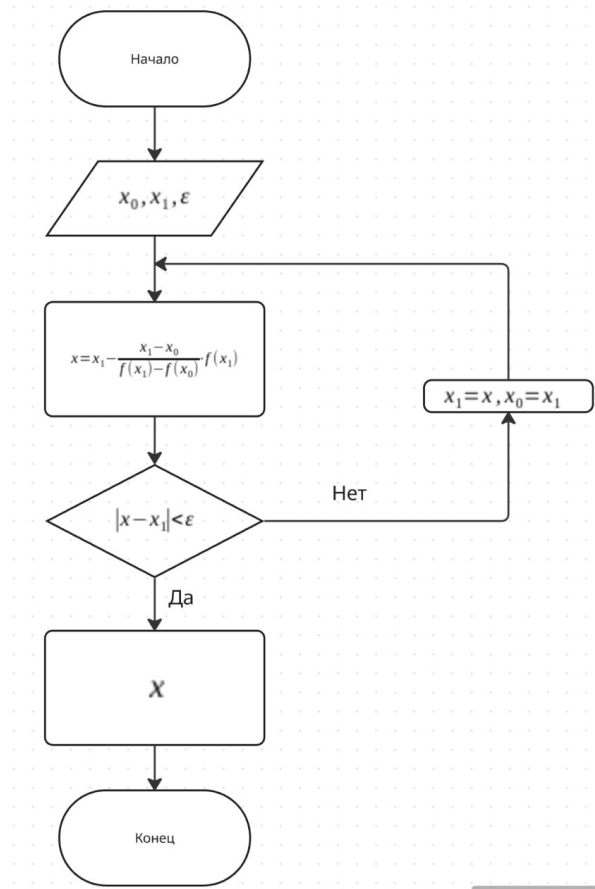
$$|x_1 - x_0| \leq \varepsilon, |y_1 - y_0| \leq \varepsilon$$

$$|-0.9546 - (-0.96)| \leq \varepsilon, |-0.2985 - (-0.28)| \leq \varepsilon \rightarrow \text{ответ найден, корень 2: } (-0.9546, -0.2985)$$

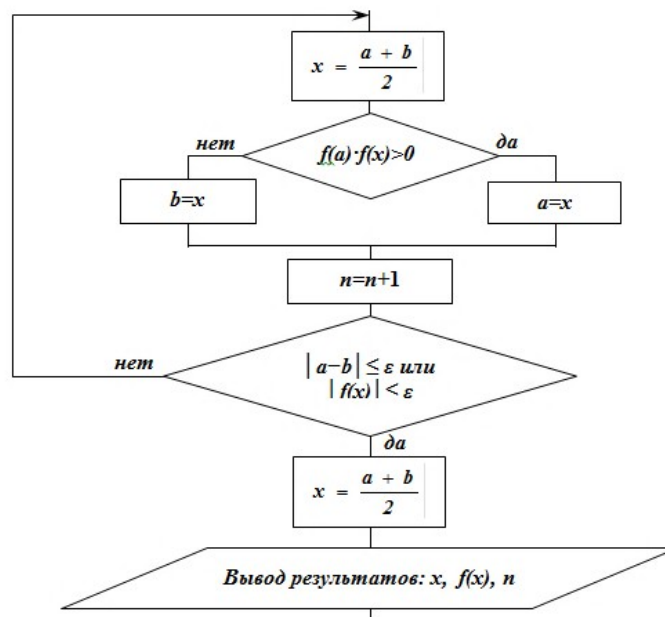
Все корни найдены.

2. Блок-схемы методов

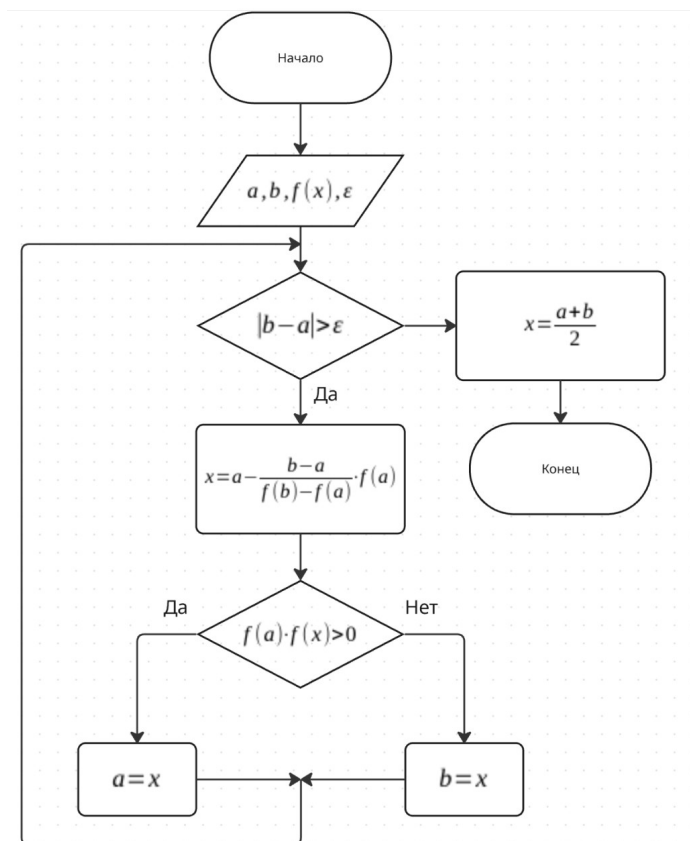
Метод секущих



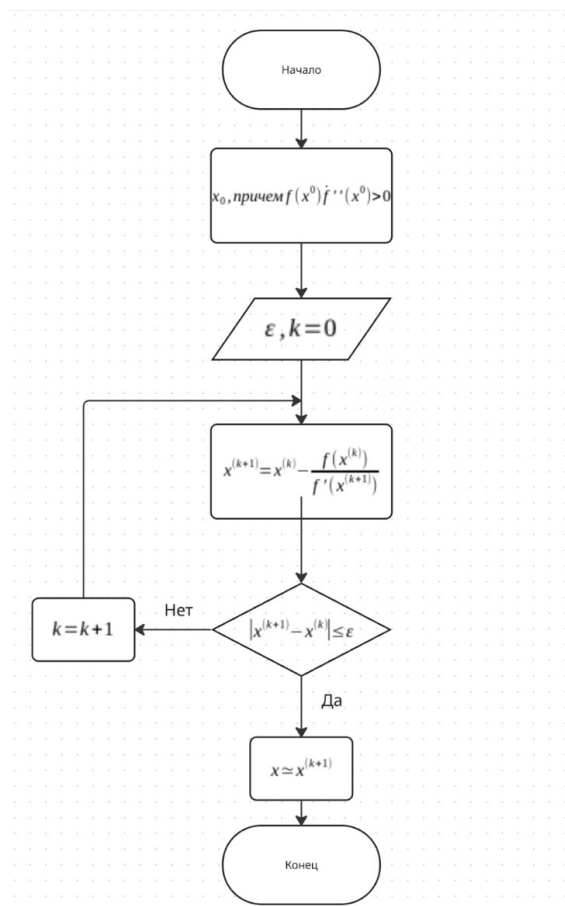
Метод половинного деления



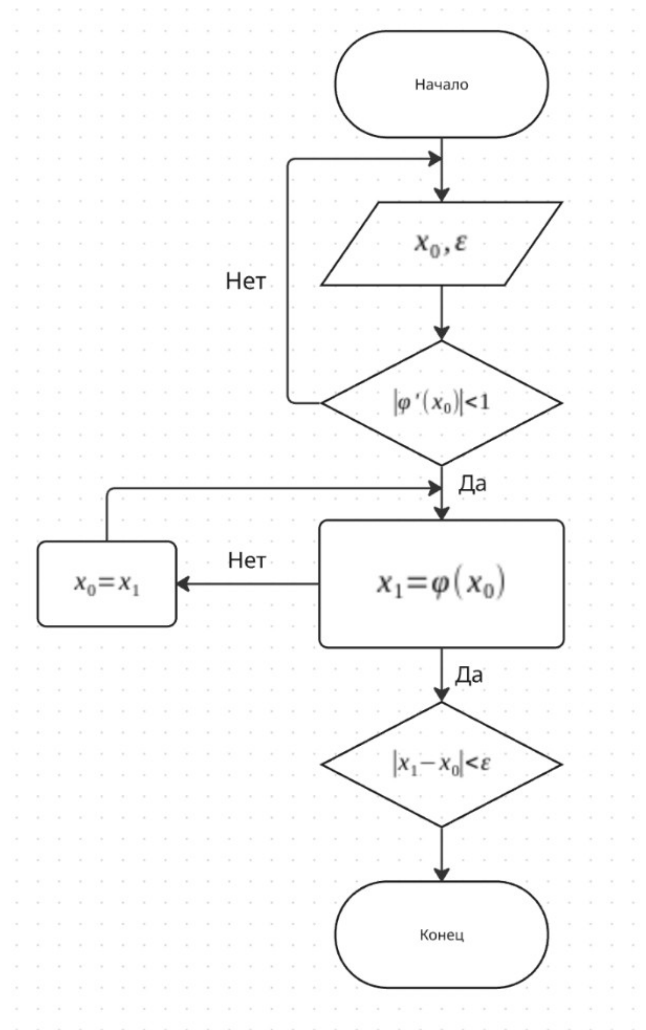
Метод хорд



Метод Ньютона



Метод простой итерации



3. Программная реализация задачи

<https://github.com/CompMathLabs/2026-p3213-lab2-yaart>

Код решений

```
@Slf4j
@Component("chord")
public class ChordSolver implements EquationSolver {
    private static final int MAX_ITERATIONS = 1000;
    private static final double EPSILON_COMPARISON = 1e-12;

    @Override
    public SolveResponse solve(Function f, double a, double b, double eps) {
        log.debug("Метод хорд на [{}, {}]", a, b);

        boolean fixLeft = f.calculate(a) * f.secondDerivative(a) > 0;
        double staticPoint = fixLeft ? a : b;
        double xCurr = fixLeft ? b : a;

        log.debug("Неподвижная точка: {}, начальное приближение: {}", staticPoint, xCurr);

        return performSolve(f, staticPoint, xCurr, eps, fixLeft);
    }

    private SolveResponse performSolve(Function f, double stat, double x0, double eps, boolean
isLeftFixed) {
        List<IterationStep> history = new ArrayList<>();
        double xCurr = x0;
        double xPrev;
        int iter = 0;

        double intervalStart = Math.min(stat, x0);
        double intervalEnd = Math.max(stat, x0);

        do {
            iter++;
            xPrev = xCurr;

            double fPrev = f.calculate(xPrev);
            double fStat = f.calculate(stat);

            if (Math.abs(fPrev - fStat) < EPSILON_COMPARISON) {
                throw new MathException(
                    "Деление на ноль в методе хорд:  $f(x) - f(stat) \approx 0$ ",
                    "DIVISION_BY_ZERO"
                );
            }

            xCurr = xPrev - fPrev * (xPrev - stat) / (fPrev - fStat);

            if (xCurr < intervalStart || xCurr > intervalEnd) {
                throw new MathException(
                    String.format("Метод хорд: приближение вышло за границы интервала [%f, %f] (x
= %f). " +
                        "Возможно, на интервале несколько корней или функция не
удовлетворяет условиям метода.",
                    intervalStart, intervalEnd, xCurr)
                );
            }
        } while (Math.abs(xCurr - xPrev) > eps);

        return new SolveResponse(xCurr, history);
    }
}
```

```

        intervalStart, intervalEnd, xCurr),
        "OUT_OF_BOUNDS"
    );
}

double aVal = isLeftFixed ? stat : Math.min(xCurr, xPrev);
double bVal = isLeftFixed ? Math.max(xCurr, xPrev) : stat;

history.add(IterationStep.builder()
    .n(iter)
    .a(aVal)
    .b(bVal)
    .x(xCurr)
    .fA(f.calculate(aVal))
    .fB(f.calculate(bVal))
    .fX(f.calculate(xCurr))
    .diff(Math.abs(xCurr - xPrev))
    .build());

} while (Math.abs(xCurr - xPrev) > eps && iter < MAX_ITERATIONS);

if (iter >= MAX_ITERATIONS) {
    throw new MathException(
        "Метод не сошелся за " + MAX_ITERATIONS + " итераций",
        "LIMIT_EXCEEDED"
    );
}

return SolveResponse.builder()
    .success(true)
    .root(xCurr)
    .functionValue(f.calculate(xCurr))
    .iterations(iter)
    .history(history)
    .build();
}
}

@Slf4j
@Component("iteration")
public class IterationSolver implements EquationSolver {
    private static final int MAX_ITERATIONS = 1000;
    private static final int CONVERGENCE_CHECK_POINTS = 100;
    private static final double INFINITY_CHECK = 1e100;
    private static final int MAX_OUT_OF_BOUNDS = 10;

    @Override
    public SolveResponse solve(Function f, double a, double b, double eps) {
        log.debug("Метод простой итерации на [{}, {}]", a, b);

        double faDeriv = f.firstDerivative(a);
        double fbDeriv = f.firstDerivative(b);
        double maxDeriv = Math.max(Math.abs(faDeriv), Math.abs(fbDeriv));

```

```

double lambda = (faDeriv > 0) ? -1.0 / maxDeriv : 1.0 / maxDeriv;

log.debug("max | f'(x) | = {}, lambda = {}", maxDeriv, lambda);

if (!checkConvergenceCondition(f, a, b, lambda)) {
    throw new MathException(
        "Условие сходимости не выполнено на интервале [" + a + ", " + b + "]. " +
        "Метод простой итерации может расходиться.",
        "DIVERGENCE_CONDITION"
    );
}

List<IterationStep> history = new ArrayList<>();

double xCurr = selectInitialApproximation(f, a, b);
double xPrev;
int iter = 0;
int outOfBoundsCount = 0;

do {
    iter++;
    xPrev = xCurr;

    double fVal = f.calculate(xPrev);
    xCurr = xPrev + lambda * fVal;

    if (Double.isInfinite(xCurr) || Double.isNaN(xCurr) || Math.abs(xCurr) > INFINITY_CHECK) {
        throw new MathException(
            String.format("Метод расходится: значение x = %f на итерации %d", xCurr, iter),
            "DIVERGENCE_INFINITY"
        );
    }

    if (xCurr < a || xCurr > b) {
        outOfBoundsCount++;
        log.warn("Приближение вышло за границы интервала: {} (раз {})", xCurr,
outOfBoundsCount);

        if (outOfBoundsCount > MAX_OUT_OF_BOUNDS) {
            throw new MathException(
                String.format("Метод расходится: последовательность выходит за границы
интервала [%f, %f] более %d раз. " +
                    "Последнее значение x = %f", a, b, MAX_OUT_OF_BOUNDS, xCurr),
                "DIVERGENCE_OUT_OF_BOUNDS"
            );
        }
    }

    history.add(IterationStep.builder()
        .n(iter)
        .x(xCurr)
        .fX(f.calculate(xCurr))
        .diff(Math.abs(xCurr - xPrev))

```

```

        .build());

log.trace("Итерация {}: x={}, diff={}", iter, xCurr, Math.abs(xCurr - xPrev));

if (iter > 5) {
    double prevDiff = history.get(history.size() - 2).getDiff();
    double currDiff = Math.abs(xCurr - xPrev);

    if (currDiff > prevDiff * 1.5 && currDiff > eps * 10) {
        log.warn("Разница начала увеличиваться: было {}, стало {}", prevDiff, currDiff);

        if (currDiff > prevDiff * 3) {
            throw new MathException(
                String.format("Метод расходится: разница между итерациями растет (было %f,
стало %f)",
                    prevDiff, currDiff),
                "DIVERGENCE_INCREASING"
            );
        }
    }
}

} while (Math.abs(xCurr - xPrev) > eps && iter < MAX_ITERATIONS);

if (iter >= MAX_ITERATIONS) {
    throw new MathException(
        "Метод не сошелся за " + MAX_ITERATIONS + " итераций",
        "LIMIT_EXCEEDED"
    );
}

if (xCurr < a || xCurr > b) {
    log.warn("Найденный корень {} находится вне исходного интервала [{}, {}]", xCurr, a, b);
}

double finalF = f.calculate(xCurr);
if (Math.abs(finalF) > eps * 100) {
    log.warn("Значение функции в корне {} слишком велико: {}", xCurr, finalF);
}

return SolveResponse.builder()
    .success(true)
    .root(xCurr)
    .functionValue(finalF)
    .iterations(iter)
    .history(history)
    .build();
}

private double findMaxDerivative(Function f, double a, double b) {
    double step = (b - a) / CONVERGENCE_CHECK_POINTS;
    double max = Math.abs(f.firstDerivative(a));

```

```

for (int i = 1; i <= CONVERGENCE_CHECK_POINTS; i++) {
    double x = a + i * step;
    double deriv = Math.abs(f.firstDerivative(x));
    if (deriv > max) {
        max = deriv;
    }
}
return max;
}

private boolean checkConvergenceCondition(Function f, double a, double b, double lambda) {
    double step = (b - a) / CONVERGENCE_CHECK_POINTS;
    double maxPhiDeriv = 0;

    for (int i = 0; i <= CONVERGENCE_CHECK_POINTS; i++) {
        double x = a + i * step;
        double phiDeriv = 1 + lambda * f.firstDerivative(x);
        double absPhiDeriv = Math.abs(phiDeriv);

        if (absPhiDeriv >= 1) {
            log.debug("Условие сходимости нарушено в x={}:  $|\varphi'(x)| = {}$ ", x, absPhiDeriv);
            return false;
        }

        if (absPhiDeriv > maxPhiDeriv) {
            maxPhiDeriv = absPhiDeriv;
        }
    }

    log.debug("Условие сходимости выполнено.  $\max |\varphi'(x)| = {}$  на интервале [{}, {}]",
        maxPhiDeriv, a, b);
    return true;
}

private double selectInitialApproximation(Function f, double a, double b) {
    double fa = f.calculate(a);
    double fb = f.calculate(b);

    if (fa * fb > 0) {
        log.warn("Функция имеет одинаковые знаки на концах интервала: f({})={}, f({})={}", a, fa,
            b, fb);
    }

    if (Math.abs(fa) < Math.abs(fb)) {
        return a;
    } else {
        return b;
    }
}

@Slf4j
@Component("secant")

```

```

public class SecantSolver implements EquationSolver {
    private static final int MAX_ITERATIONS = 1000;
    private static final double EPSILON_COMPARISON = 1e-12;

    @Override
    public SolveResponse solve(Function f, double a, double b, double eps) {
        log.debug("Метод секущих на [{}, {}]", a, b);

        List<IterationStep> history = new ArrayList<>();

        double xPrev = a;
        double xCurr = b;
        int iter = 0;

        do {
            iter++;

            double fCurr = f.calculate(xCurr);
            double fPrev = f.calculate(xPrev);

            if (Math.abs(fCurr - fPrev) < EPSILON_COMPARISON) {
                throw new MathException(
                    "Деление на ноль в методе секущих:  $f(x_k) - f(x_{k-1}) \approx 0$ ",
                    "DIVISION_BY_ZERO"
                );
            }

            double xNext = xCurr - fCurr * (xCurr - xPrev) / (fCurr - fPrev);

            xPrev = xCurr;
            xCurr = xNext;

            history.add(IterationStep.builder()
                .n(iter)
                .x(xCurr)
                .fX(f.calculate(xCurr))
                .diff(Math.abs(xCurr - xPrev))
                .build());

            log.trace("Итерация {}: x={}, diff={}", iter, xCurr, Math.abs(xCurr - xPrev));
        } while (Math.abs(xCurr - xPrev) > eps && iter < MAX_ITERATIONS);

        if (iter >= MAX_ITERATIONS) {
            throw new MathException(
                "Метод не сошелся за " + MAX_ITERATIONS + " итераций",
                "LIMIT_EXCEEDED"
            );
        }

        return SolveResponse.builder()
            .success(true)
            .root(xCurr)

```

```

        .functionValue(f.calculate(xCurr))
        .iterations(iter)
        .history(history)
        .build();
    }
}
@Slf4j
@Component
public class SystemIterationSolver {
    private static final int MAX_ITERATIONS = 1000;
    public SystemSolveResponse solve(EquationSystem system, double x0, double y0, double eps) {
        log.info("Начало решения системы: x0={}, y0={}, eps={}", x0, y0, eps);

        boolean converges = checkConvergence(system, x0, y0);
        if (!converges) {
            log.warn("Достаточное условие сходимости не выполнено, но пробуем продолжить");
        }

        List<SystemIterationStep> history = new ArrayList<>();
        double xCurr = x0;
        double yCurr = y0;
        double xPrev, yPrev;
        int iter = 0;
        double maxDiff;

        do {
            iter++;
            xPrev = xCurr;
            yPrev = yCurr;

            try {
                double[] nextValues = system.calculate(new double[]{xPrev, yPrev});
                xCurr = nextValues[0];
                yCurr = nextValues[1];
            } catch (Exception e) {
                throw new MathException(
                    "Ошибка при вычислении на итерации " + iter + ": " + e.getMessage(),
                    "CALCULATION_ERROR"
                );
            }

            if (Double.isNaN(xCurr) || Double.isInfinite(xCurr) ||
                Double.isNaN(yCurr) || Double.isInfinite(yCurr)) {
                throw new MathException(
                    "Метод расходится (получены бесконечные значения)",
                    "DIVERGENCE"
                );
            }

            double deltaX = Math.abs(xCurr - xPrev);
            double deltaY = Math.abs(yCurr - yPrev);
            maxDiff = Math.max(deltaX, deltaY);

```

```

        history.add(SystemIterationStep.builder()
            .n(iter)
            .x(xCurr)
            .y(yCurr)
            .deltaX(deltaX)
            .deltaY(deltaY)
            .build());

        log.debug("Итерация {}: x={}, y={}, deltaX={}, deltaY={}", iter, xCurr, yCurr, deltaX, deltaY);

    } while (maxDiff > eps && iter < MAX_ITERATIONS);

    if (iter >= MAX_ITERATIONS) {
        throw new MathException(
            "Метод не сошелся за " + MAX_ITERATIONS + " итераций",
            "LIMIT_EXCEEDED"
        );
    }

    double[][] residuals = system.originalFunctions(new double[]{xCurr, yCurr});
    double residualF = Math.abs(residuals[0][0]);
    double residualG = Math.abs(residuals[1][0]);

    log.info("Решение найдено за {} итераций: x={}, y={}", iter, xCurr, yCurr);
    log.info("Невязки: |f|={}, |g|={}", residualF, residualG);

    return SystemSolveResponse.builder()
        .success(true)
        .resultVector(new double[]{xCurr, yCurr})
        .errorVector(new double[]{
            history.get(history.size() - 1).getDeltaX(),
            history.get(history.size() - 1).getDeltaY()
        })
        .residuals(new double[]{residualF, residualG})
        .iterations(iter)
        .history(history)
        .build();
}

private boolean checkConvergence(EquationSystem system, double x, double y) {
    try {
        double[][] j = system.jacobian(new double[]{x, y});

        double row1 = Math.abs(j[0][0]) + Math.abs(j[0][1]);
        double row2 = Math.abs(j[1][0]) + Math.abs(j[1][1]);

        boolean converges = row1 < 1 && row2 < 1;
        log.debug("Проверка сходимости: row1={}, row2={}, сходится={}", row1, row2, converges);

        return converges;
    } catch (Exception e) {
        log.warn("Не удалось вычислить матрицу Якоби для проверки сходимости", e);
        return false;
    }
}

```

}

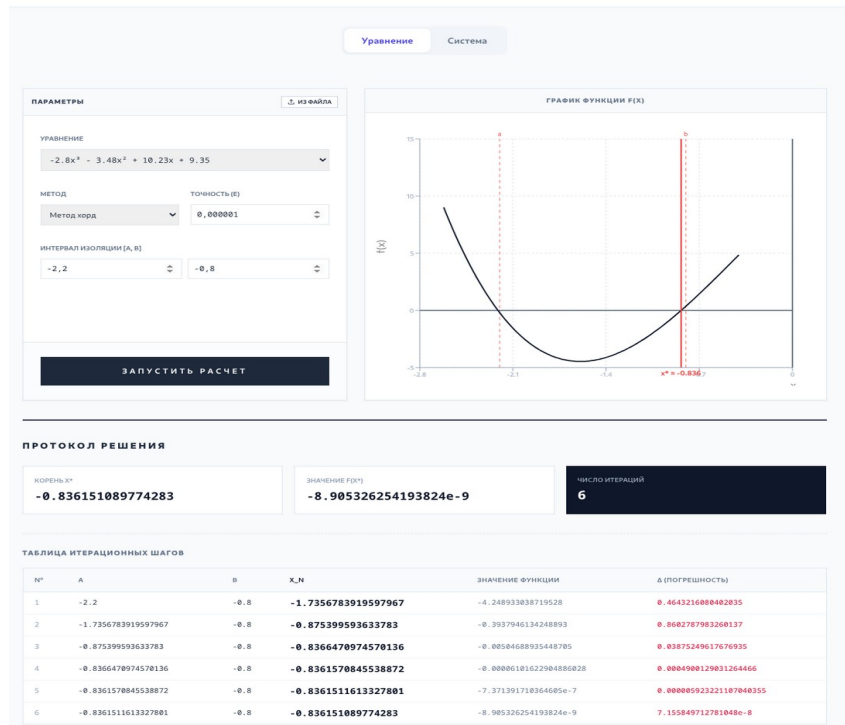
}

}

Примеры работы программы:

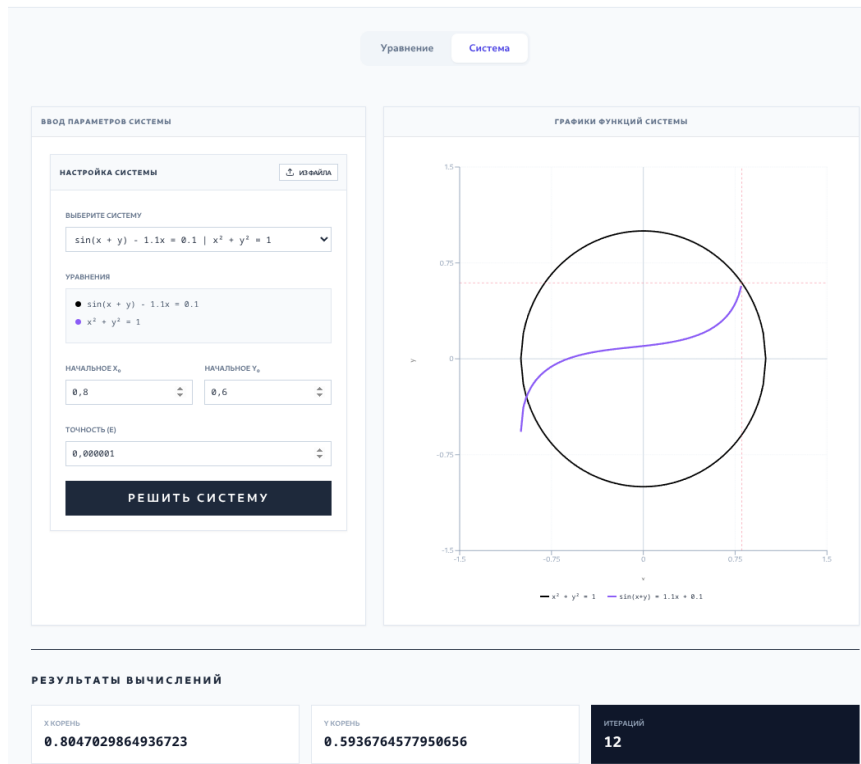
Лабораторная работа №2
ЧИСЛЕННЫЕ МЕТОДЫ РЕШЕНИЯ НУ И СЛУ

Ясаков А. А. | Р3213
ВАРИАНТ 20 | ИСУ-468108



Лабораторная работа №2
ЧИСЛЕННЫЕ МЕТОДЫ РЕШЕНИЯ НУ И СЛУ

Ясаков А. А. | Р3213
ВАРИАНТ 20 | ИСУ-468108



Вывод

В ходе выполнения лабораторной работы были изучены численные методы решения нелинейных уравнений и систем нелинейных уравнений с использованием Spring+React. В результате работы были найдены корни заданных уравнений и систем с использованием различных численных методов, а также были построены графики функций для полного представления исследуемых интервалов. Проведен сравнительный анализ пяти итерационных методов. Установлено, что выбор метода напрямую зависит от свойств функции и требуемой вычислительной мощности:

Метод половинного деления: является самым надежным, так как сходится для любой непрерывной функции, имеющей разные знаки на концах отрезка. Однако он обладает самой низкой (линейной) скоростью сходимости. Для достижения высокой точности требуется

наибольшее количество итераций $n \approx \log_2 \left(\frac{(b-a)}{e} \right)$

$$x_i = \frac{a_i + b_i}{2}$$

Метод хорд: использует линейную интерполяцию. В отличие от метода половинного деления, он учитывает значения функции на границах, что обычно ускоряет процесс. Скорость сходимости выше линейной, но метод может замедляться, если одна из границ отрезка остается неподвижной.

$$x_i = a_i - \frac{b_i - a_i}{f(b_i) - f(a_i)} \cdot f(a_i)$$

Метод Ньютона (касательных): продемонстрировал самую высокую скорость сходимости. Метод требует вычисления производной $f'(x)$ и очень чувствителен к выбору начального приближения x_0 . При неудачном выборе точки или вблизи экстремумов метод может разойтись.

$$x_i = x_{i-1} - \frac{f(x_{i-1})}{f'(x_{i-1})}$$

Метод секущих: является модификацией метода Ньютона, где производная заменяется разностным отношением. Он не требует знания аналитического вида производной, сохраняя при этом скорость сходимости, близкую к квадратичной, что делает его одним из самых практичных методов.

$$x_{i+1} = x_i - \frac{x_i - x_{i-1}}{f(x_i) - f(x_{i-1})} \cdot f(x_{i-1})$$

Метод простой итерации (МПИ): наиболее элегантен в реализации, но требует предварительного приведения уравнения к виду $x = \varphi(x)$. Главный недостаток - необходимость соблюдения жесткого условия сходимости $|\varphi'(x)| < 1$. Если условие выполнено, метод сходится со скоростью геометрической прогрессии.

$$x_{i+1} = \varphi(x_i)$$

Общий итог: Для систем нелинейных уравнений наиболее универсальным оказался метод Ньютона (в силу его скорости), а для одиночных уравнений при наличии интервала изоляции - метод секущих, как оптимальный баланс между простотой вычислений и скоростью сходимости.